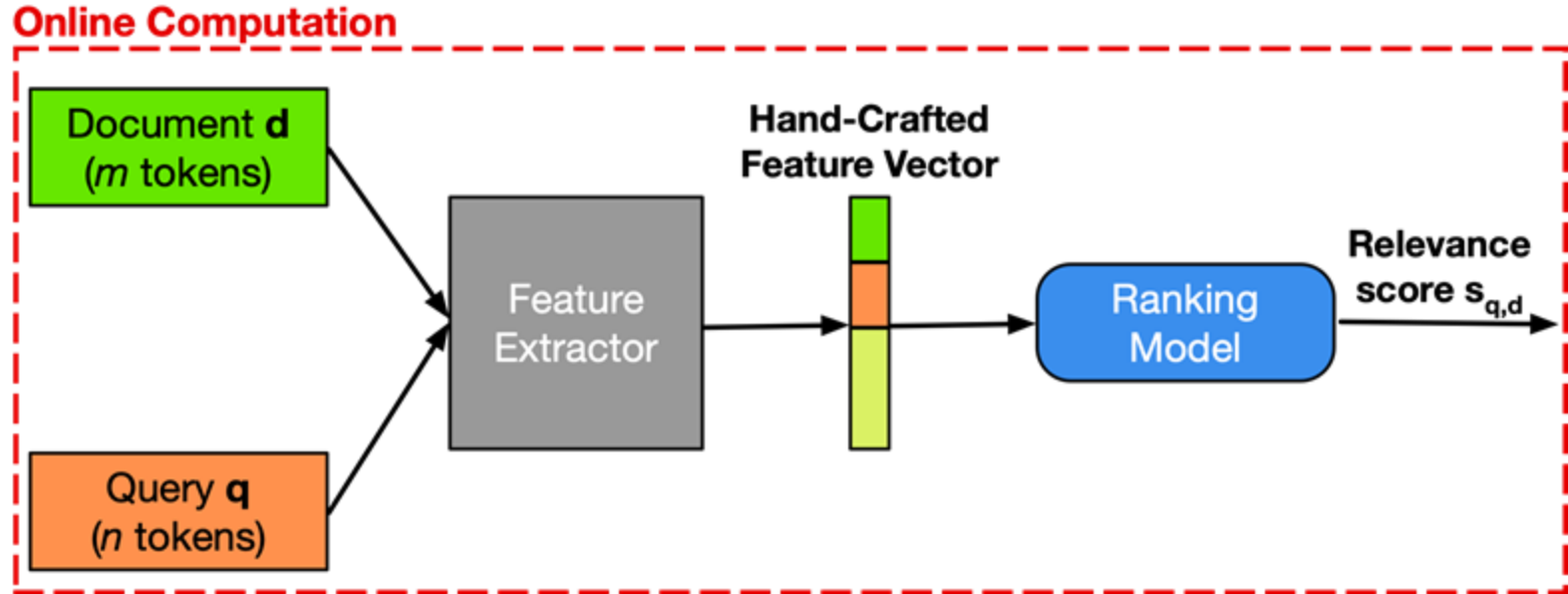


LEARNING TO RANK

Machine Learning for IR on Hand-Crafted Features



- Input: **vector of features** modeling relevance for query/doc + **labels**
- State-of-the-art approach: forest of **thousands** of regression trees
- **High quality**, but their use is **computationally expensive**!

Is it an easy or difficult task?

Learn the ranking, not the label.

Complex, because ranking quality measures are not nicely derivable and **gradient descent not applicable**.

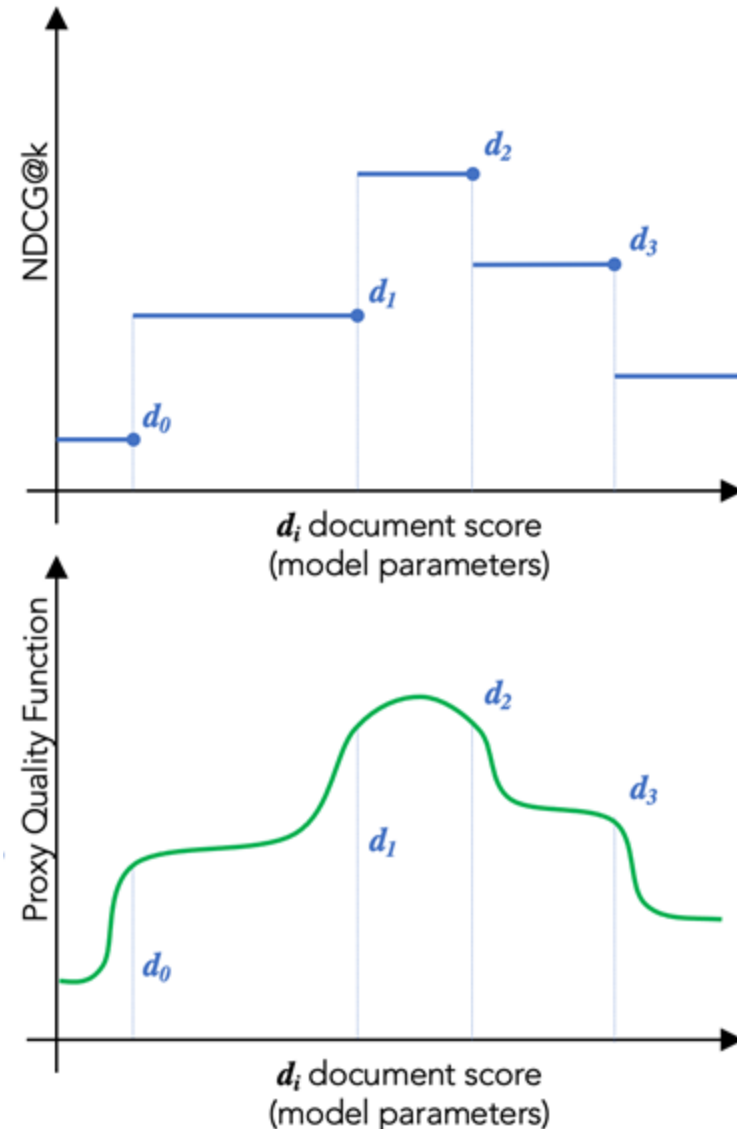
Huge datasets, hundreds of features.

Rank-based measures (NDCG, ERR, MAP, ...) depend on **documents sorted order**

- **gradient is either 0** (sorted order did not change) or **undefined** (discontinuity)

Solution: we need a proxy Loss function

- it should be **differentiable**
- and with a **similar behavior of the original cost function**



Point-Wise Algorithms

Each document is considered independently from the others

- No information about other candidates for the same query is used at training time

A different cost-function is optimized

- Several approaches:
 - Regression
 - Multi-Class Classification
 - Ordinal regression

Among Regression-Based:

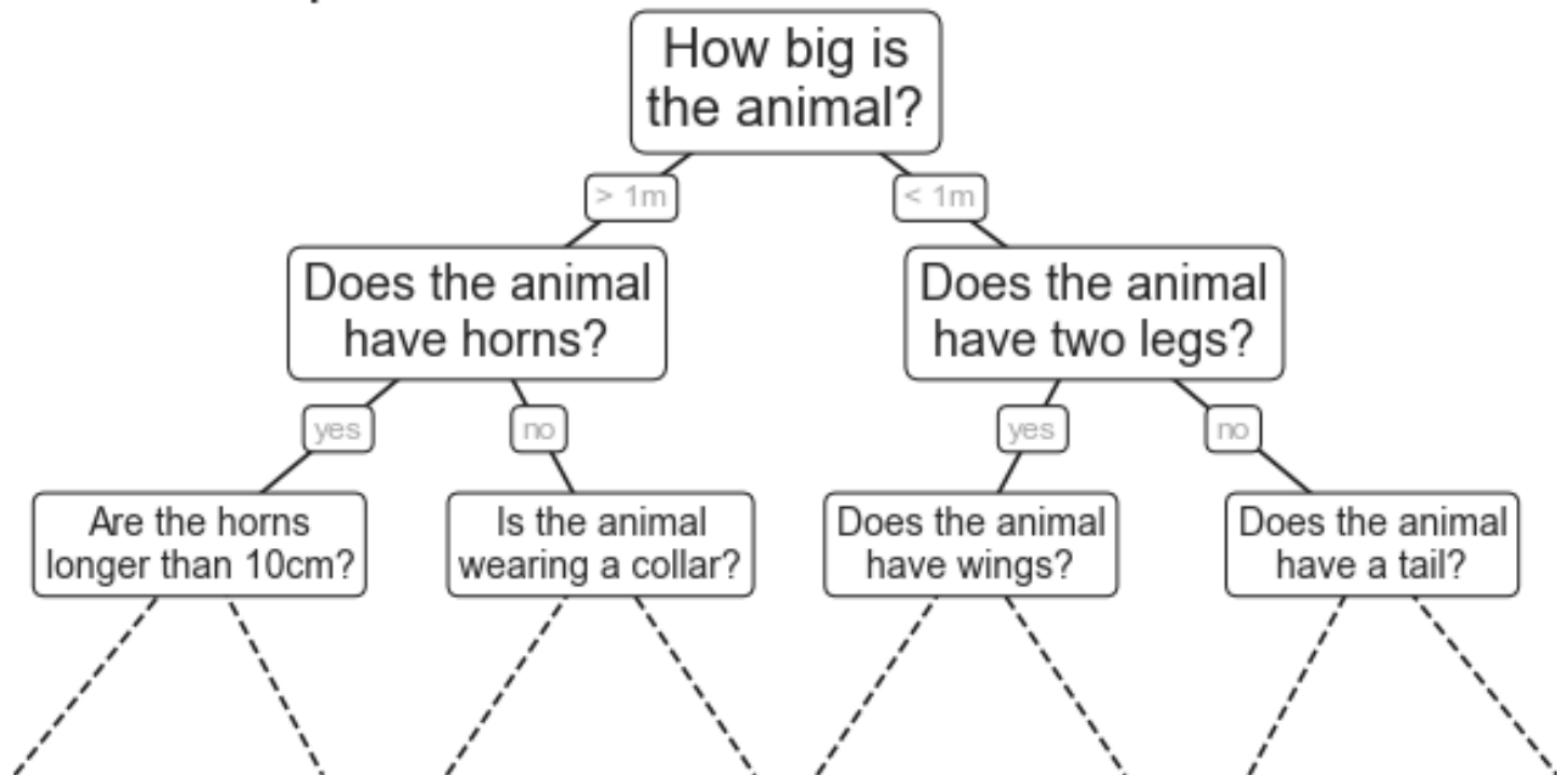
Gradient Boosting Regression Trees

- **Sum of Squared Errors (SSE)** is minimized



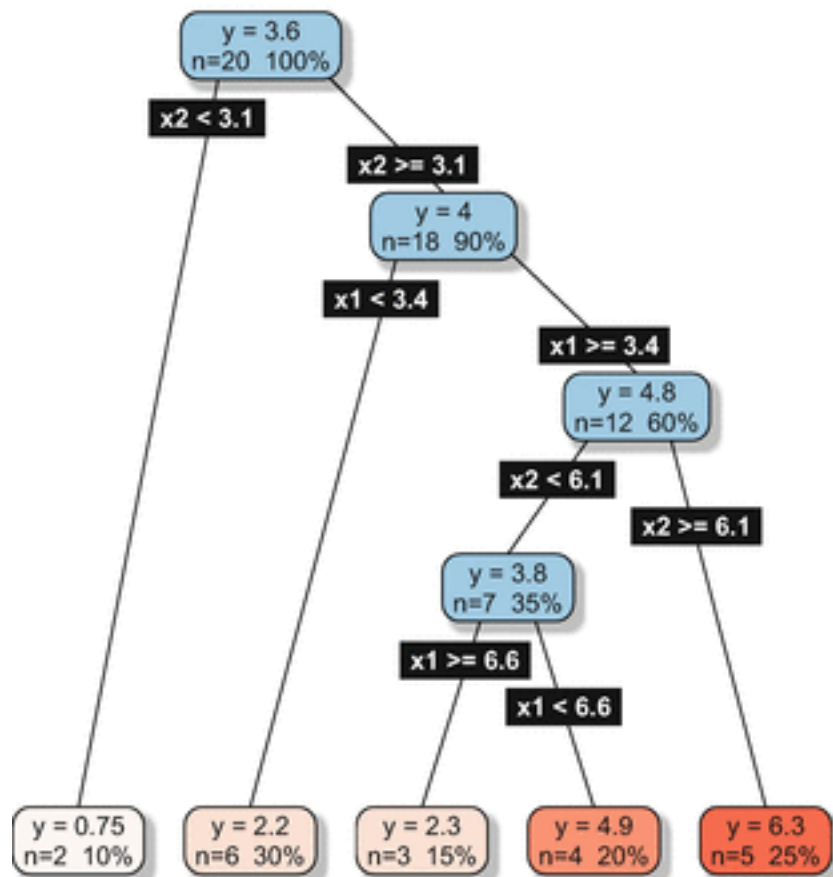
Decision Tree

Example Decision Tree: Animal Classification

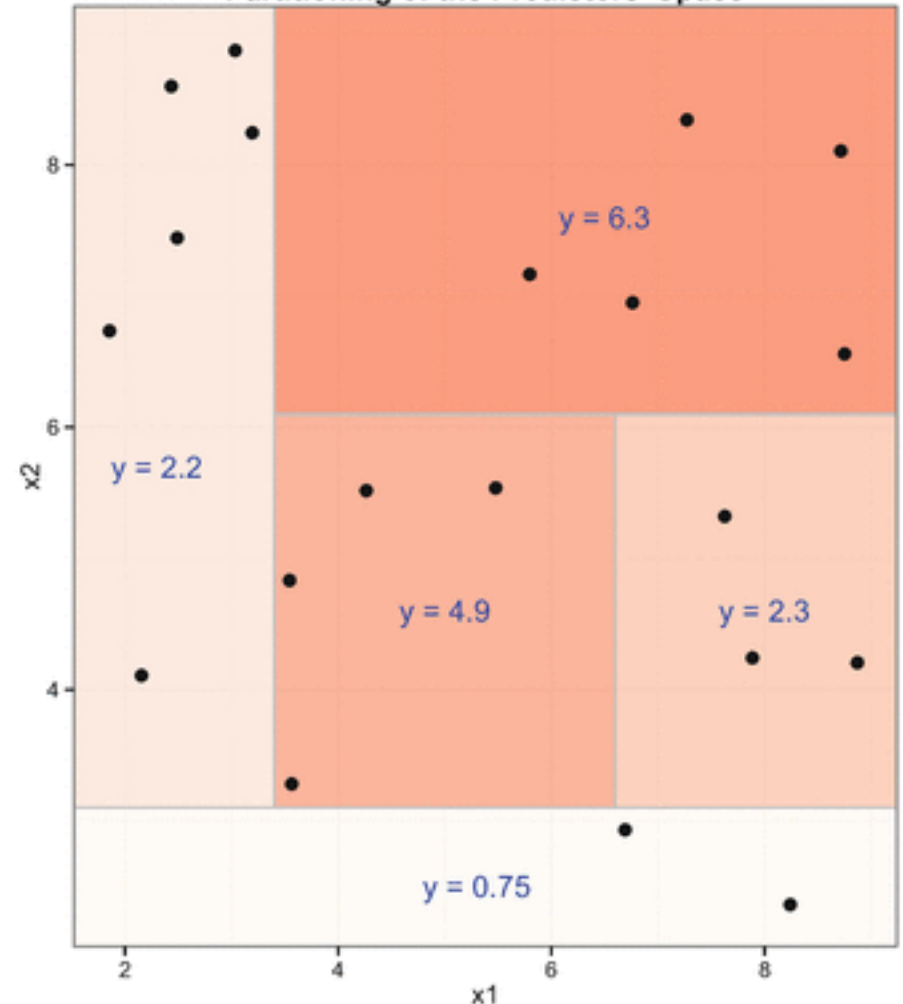


Regression Trees

Example of a Regression Tree



Partitioning of the Predictors' Space



Gradient Boosting Regression Trees

- **Tree forests**: GBRT, Lambda MART, Random Forest, Oblivious Trees, etc.
 - ensemble of **weak learners**, each contributing a partial score
 - at scoring time, all trees can be **processed independently**



Gradient Boosting Regression Trees

Iterative algorithm: $F(d) = \sum_i f_i(d)$ Weak Learner

Each f_i is regarded as a step in the best optimization direction, i.e., a **steepest descent step**:

by line-search

$$f_i(d) = -\rho_i g_i(d)$$

negative gradient

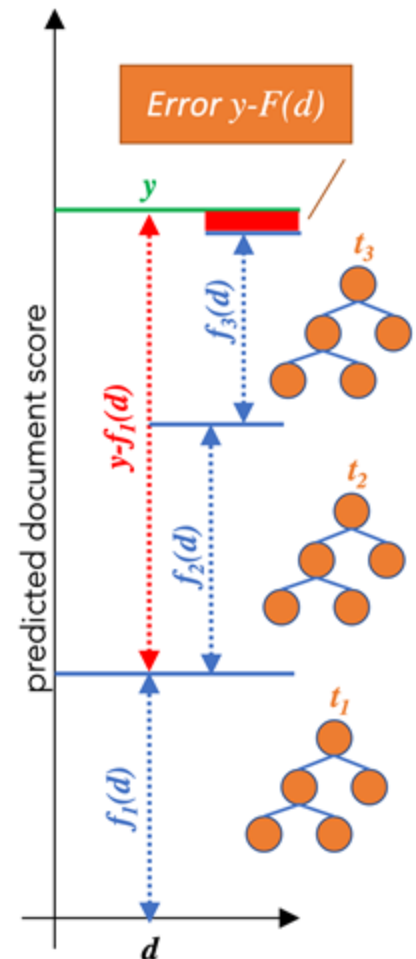
$$-g_i(d) = -\left[\frac{\partial L(y, f(d))}{\partial f(d)} \right]_{f=\sum_{j<i} f_j}$$

Given $L = SSE/2$:

$$-\frac{\partial[\frac{1}{2}SSE(y, f(d))]}{\partial f(d)} = -\frac{\partial[\frac{1}{2} \sum (y - f(d))^2]}{\partial f(d)} = y - f(d)$$

pseudo-response

Gradient g_i is approximated by a Regression Tree t_i



Pair-wise Algorithms (RankNet)

Documents are considered in pairs

Estimated probability that d_i is better than d_j is:

$$P_{ij} = \frac{e^{o_{ij}}}{1 + e^{o_{ij}}}$$

$$o_{ij} = F(d_i) - F(d_j)$$

Let T_{ij} be the true probability, the **Cross Entropy Loss** is:

$$C_{ij} = -T_{ij} \log P_{ij} - (1 - T_{ij}) \log(1 - P_{ij})$$

We consider **only pairs where d_i is better than d_j , i.e., $y_i > y_j$:**

If $o_{ij} \rightarrow +\infty$
(i.e., correctly ordered)
 $C_{ij} \rightarrow 0$

$$C_{ij} = \log(1 + e^{-o_{ij}})$$

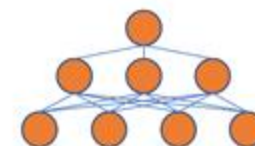
If $o_{ij} \rightarrow -\infty$
(i.e., mis-ordered)
 $C_{ij} \rightarrow +\infty$

This is **differentiable**: used to train a **Neural Network with back-propagation**.

Other approaches: Ranking-SVM, RankBoost, ...



Training Algo: ANN
Loss: Cross Entropy



Pair-wise Algorithm

RankNet performs better than other pairwise algorithms

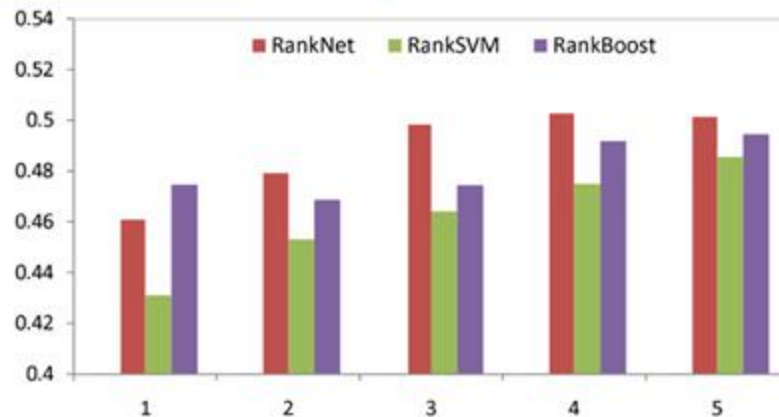


Figure 1. Ranking accuracies in terms of NDCG@n on TREC

RankNet cost is not nicely correlated with NDCG quality

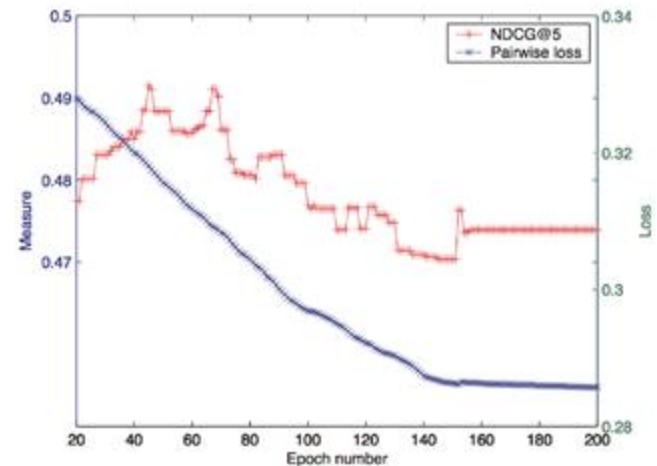


Figure 4. Pairwise loss v.s. NDCG@5 in RankNet

List-wise Algorithm (LambdaMart)

Recall: **GBRT** requires a gradient g_i for every d_i

First: **estimate the gradient comparing to d_j** , with $y_i > y_j$:

derivative of the negative RankNet cost

$\Delta \text{Quality}$ after swapping d_i with d_j

$$\lambda_{ij} = \frac{1}{1 + e^{o_{ij}}} |\Delta \text{NDCG}| = -\lambda_{ji}$$

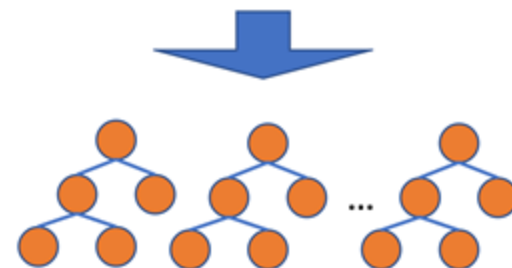
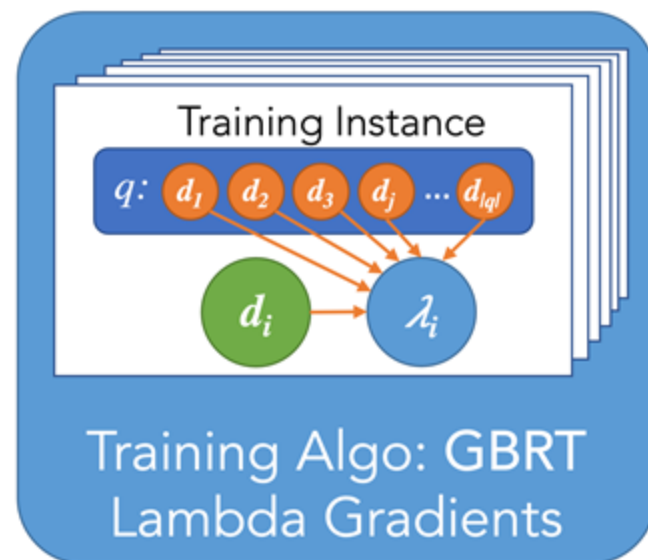
If $o_{ij} \rightarrow +\infty$
(i.e., correctly ordered)
 $\lambda_{ij} \rightarrow 0$

If $o_{ij} \rightarrow -\infty$
(i.e., mis-ordered)
 $\lambda_{ij} \rightarrow |\Delta \text{NDCG}|$

Top documents are more relevant !

Then: **estimate the gradient comparing to every other d_j for q**

$$g_i = \lambda_i = \sum_{y_i > y_j} \lambda_{ij} - \sum_{y_i < y_j} \lambda_{ij}$$



List-wise Algorithms: some results

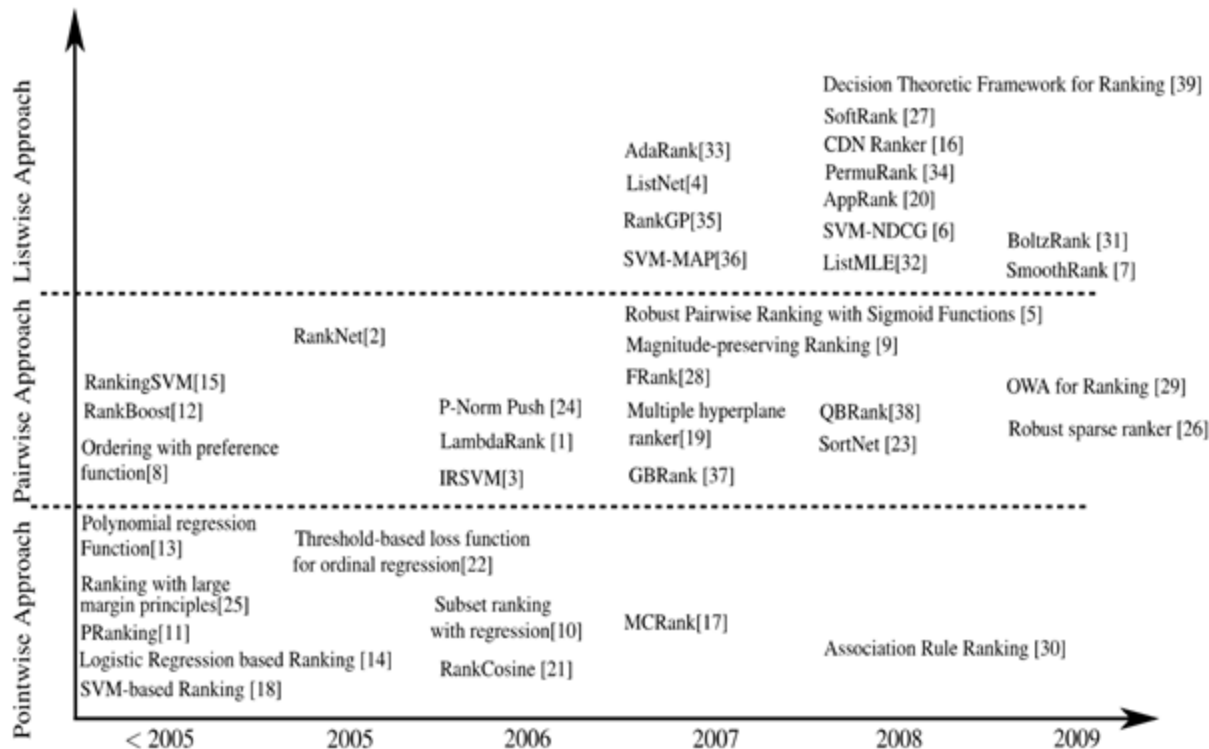


- NDCG@10 on public LtR Datasets

<i>Algorithm</i>	<i>MSN10K</i>	<i>Y!S1</i>	<i>Y!S2</i>	<i>Istella-S</i>
RankingSVM	0.4012	0.7238	0.7306	N/A
GBRT	0.4602	0.7555	0.7620	0.7313
LambdaMART	0.4618	0.7529	0.7531	0.7537

Other approaches: ListNet/ListMLE, Approximate Rank, SVM AP, RankGP, others ...

LtR Timeline



- New approaches to **optimize IR measures**:
 - DirectRank
 - LambdaMart
 - BLMart
 - SSLambdaMART
 - CoList
 - LogisticRank
 - LambdaLoss
- **Deep Learning** to improve query-document matching:
 - Conv.DNN
 - DSSM
 - Dual-Embedding
 - Local and Distributed repr.
 - Weak Supervision
 - Neural Click Model
- **On-line learning**:
 - Multi-armed bandits
 - Dueling bandits
 - K-armed dueling bandits
 - online learning

GBRT

- Successful in several *Data Challenges*:
 - Winner of the *Yahoo! LtR Challenge*: combination of 12 ranking models, 8 of which were Lambda-MART models, each having up to 3,000 trees [CC11]
 - According to the 2015 statistics, GBRTs were adopted by the majority of the winning solutions among the *Kaggle* competitions, even more than the popular deep networks, and all the top-10 teams qualified in the *KDD Cup 2015* used GBRT-based algorithms [CG16]
- New interesting *open-source implementations*:
 - XGBoost, LightGBM by *Microsoft*, CatBoost by *Yandex*
- *Pluggable* within *Apache Lucene/Solr*
 - <https://www.techatbloomberg.com/blog/bloomberg-integrated-learning-rank-apache-solr/>

Are DNN faster than Regression Forests?

- **Rationale:**

- Trust accuracy of *LambdaMART*, use a “similar” DNN at run time

- **Methodology:**

- *Ranking Distillation* [TW18]
- **Train a ANN that approximates the output of *LambdaMART***
 - ... rather than the training labels!
 - Enrich the dataset with points around discontinuities, i.e., trees’ split points
 - Networks used: Fully connected 4 layers 2000x500x500x100 and 2 layers 500x100

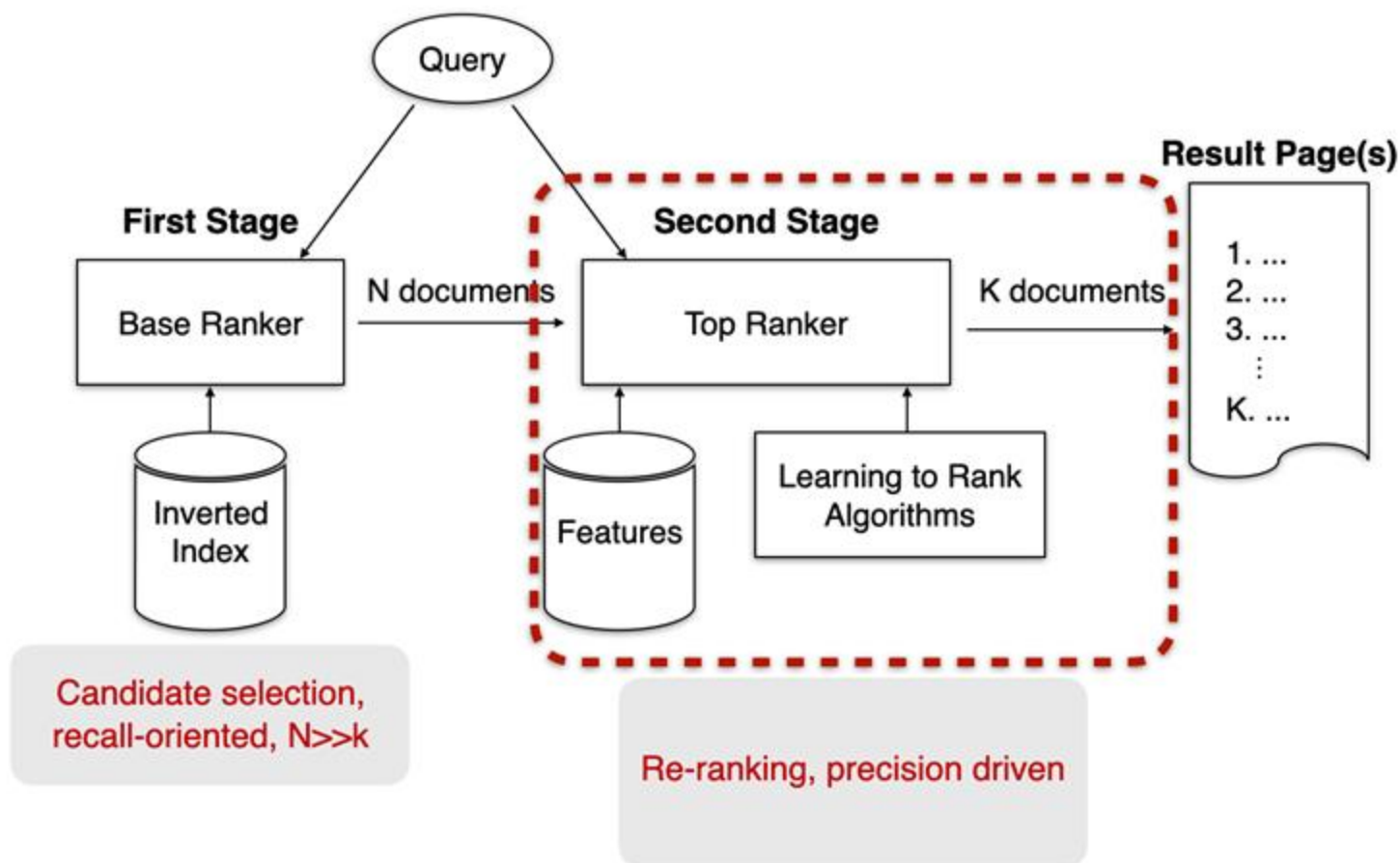
Method	# Layers	MSN30k MAP	GOV2 MAP
Regression Forest	-	0.6004	0.2995
N_{approx}	4	0.5950	0.2995
N_{approx}	2	0.5955	0.3007
$N_{\text{relevance}}$	4	0.5639*	0.2531*

Impl	Source	1000 Trees		20,000 Trees	
		8 Leaves	64 Leaves	8 Leaves	64 Leaves
Generated C++ for Forest	If-Then-Else	8.2-10.3	55.9-55.1	709.0-772.2	4462.0-4809.0
Tensorflow-CPU 4-layer	<i>N</i>				51.1-53.3
Tensorflow-CPU 2-layer	<i>N</i>				5.83-7.04
PyTorch GPU 4-layer	<i>N</i>				0.976-1.01
PyTorch GPU 2-layer	<i>N</i>				0.323-0.335



Multi-Stage Ranking

The trick of Pipelined Ranking Architecture for top-k Retrieval



Single-Stage Ranking



Requires to apply the learnt *model* to *every matching document*, and to generate the required *features*.

Not feasible!

We have at least **3** *efficiency vs. effectiveness trade-offs*.

Single-Stage Ranking



① *Feature Computation Trade-off*

- Computationally **Expensive** & highly discriminative features vs. computationally **Cheap** & slightly discriminative features

Two-Stage Ranking



Expensive features are computed only for the *top-K candidate documents* passing the first stage. **How to choose K ?**

② *Number of Matching Candidates Trade-off* :

- a *Large set* of candidates is **Expensive** and produces **high-quality** results vs. a *Small set* of candidates is **Cheap** and produces **low-quality** results
 - 1000 documents (Gov2, ClueWeb09-B collections)
 - 1500-2000 documents (ClueWeb09-B)
 - “hundreds of thousands” (over “hundreds of machines”)

Multi-Stage Ranking



- 3 stages: *Contextual features* are considered in the 3rd stage
 - Contextual => *about the current result set*
 - Rank based on specific features, Mean, Variance, Standardized features, topic model similarity
 - First two stages are executed at each serving node
- N stages: Which *model* in each stage? Which *features*? How many *documents*?
 - About *200 configurations tested*, best results with $N=3$ stages, 2500 and 700 docs between stages
- Predict the best k for STAGE 1, the best processing pipeline, learn the pipeline at training time

Multi-Stage Ranking



③ Model Complexity Trade-off :

- **Complex** & **Slow** high-quality vs. **Simple** & **Fast** low-quality models:
 - **Complex** as: Random Forest, GBRT, Initialized GBRT, Lambda-MART,
 - **Simple** as: Coordinate Ascent, Ridge Regression, SVM-Rank, RankBoost
 - **In-between** as: Oblivious Lambda-Mart, ListNet